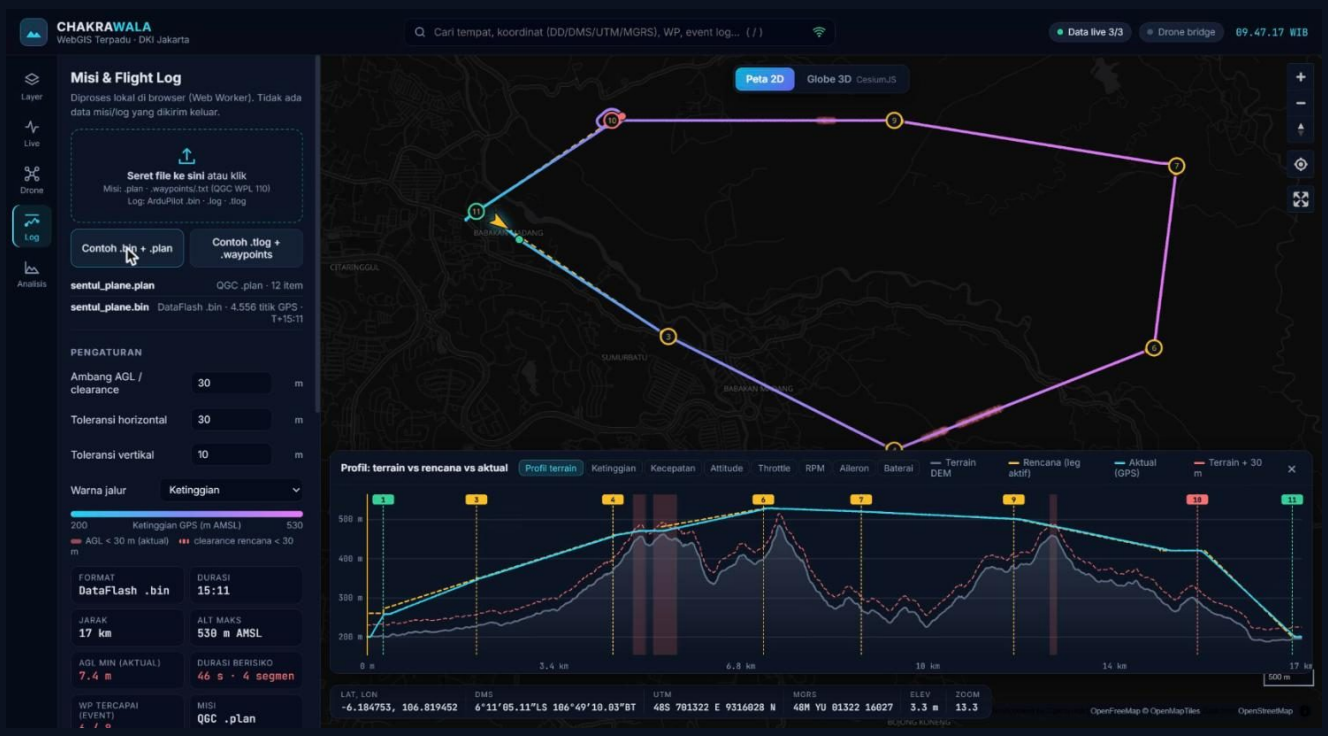
CHAKRAWALA

Panduan Pengguna — WebGIS Terpadu, Analisis Misi & Flight Log Drone



Dihasilkan otomatis dari docs/USER_GUIDE.md dan README.md.
Semua screenshot memakai data demo SINTETIS — bukan data penerbangan nyata.

Repositori: github.com/songohaji99/gcs-terrain-log-analyzer

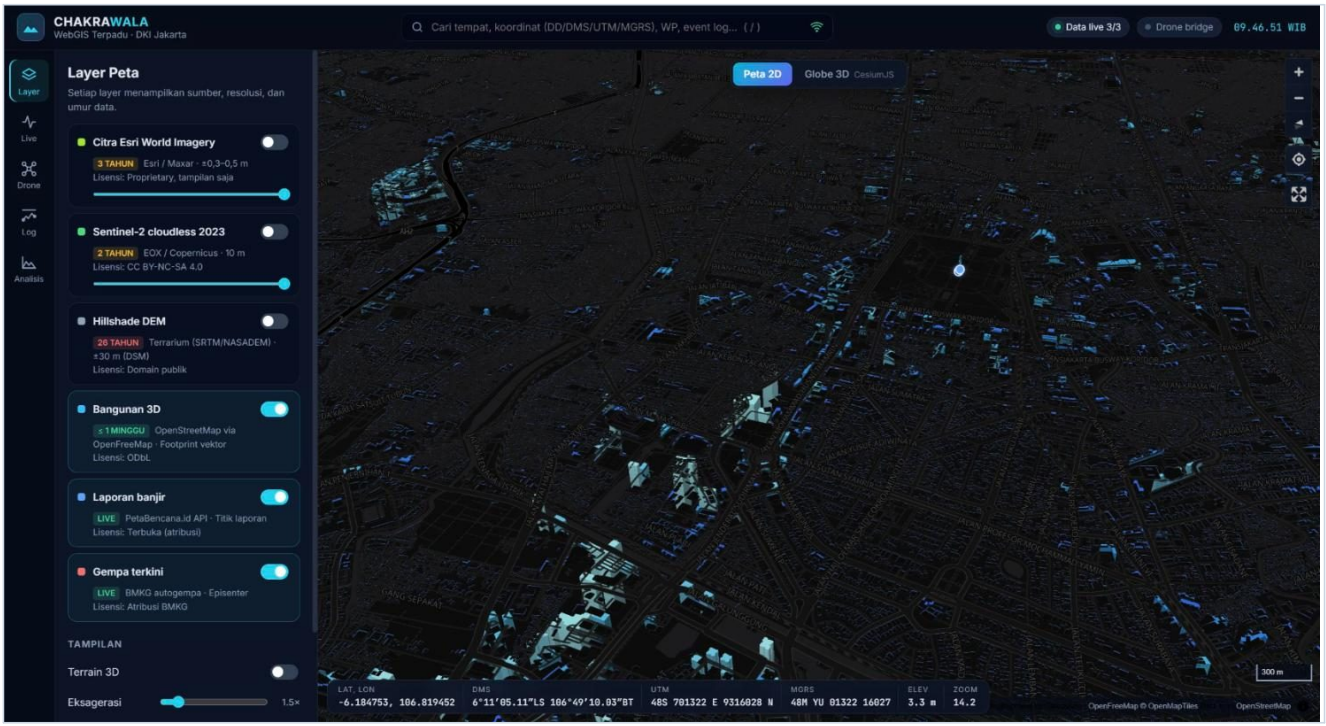
Panduan Pengguna Chakrawala

Panduan langkah demi langkah memakai **data demo sintetis** yang ikut dalam repositori (`web/public/demo/`). Jalankan aplikasi lebih dulu (`npm run setup` lalu `npm run dev`) dan buka http://localhost:5173.

Langkah 1 — Mengenal tampilan

Saat dibuka, peta 2D menampilkan Jakarta Pusat dengan bangunan 3D. Rail ikon di kiri berisi menu **Layer, Live, Drone, Log, Analisis**; klik ikon yang aktif sekali lagi untuk menutup panel. Bar bawah menampilkan koordinat kursor dalam DD, DMS, UTM, MGRS, dan elevasi DEM.

1. Buka http://localhost:5173.
2. Tunggu chip **Data live 3/3** di kanan atas menyala hijau.
3. Arahkan kursor ke peta untuk membaca koordinat.

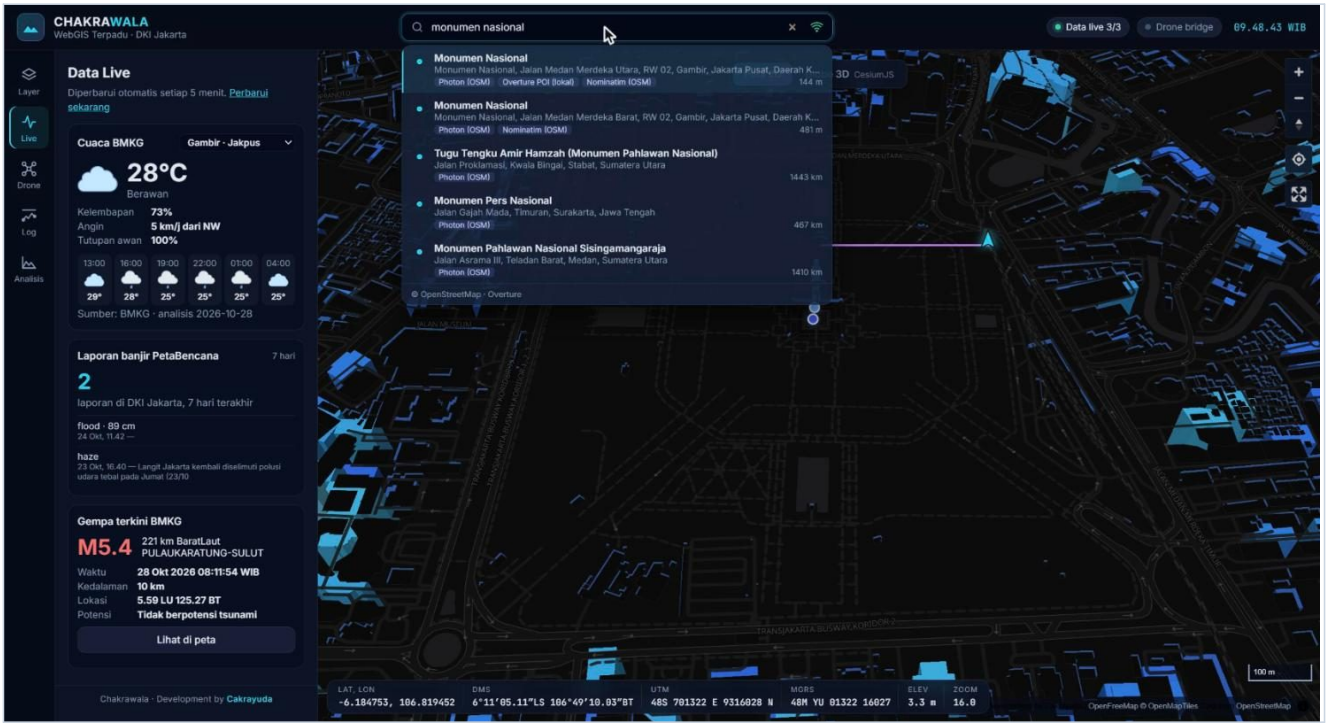


Tampilan awal Chakrawala

Langkah 2 — Data live & pencarian tempat

Menu **Live** menampilkan cuaca BMKG, laporan banjir PetaBencana, dan gempa terkini BMKG. Kotak pencarian di atas menggabungkan beberapa provider dan menampilkan sumber setiap hasil.

1. Klik ikon **Live**.
2. Klik kotak pencarian (atau tekan `/`), ketik `monumen nasional`, lalu tekan `Shift+Enter` untuk pencarian penuh.
3. Gunakan `↑/↓` dan `Enter` untuk memilih.

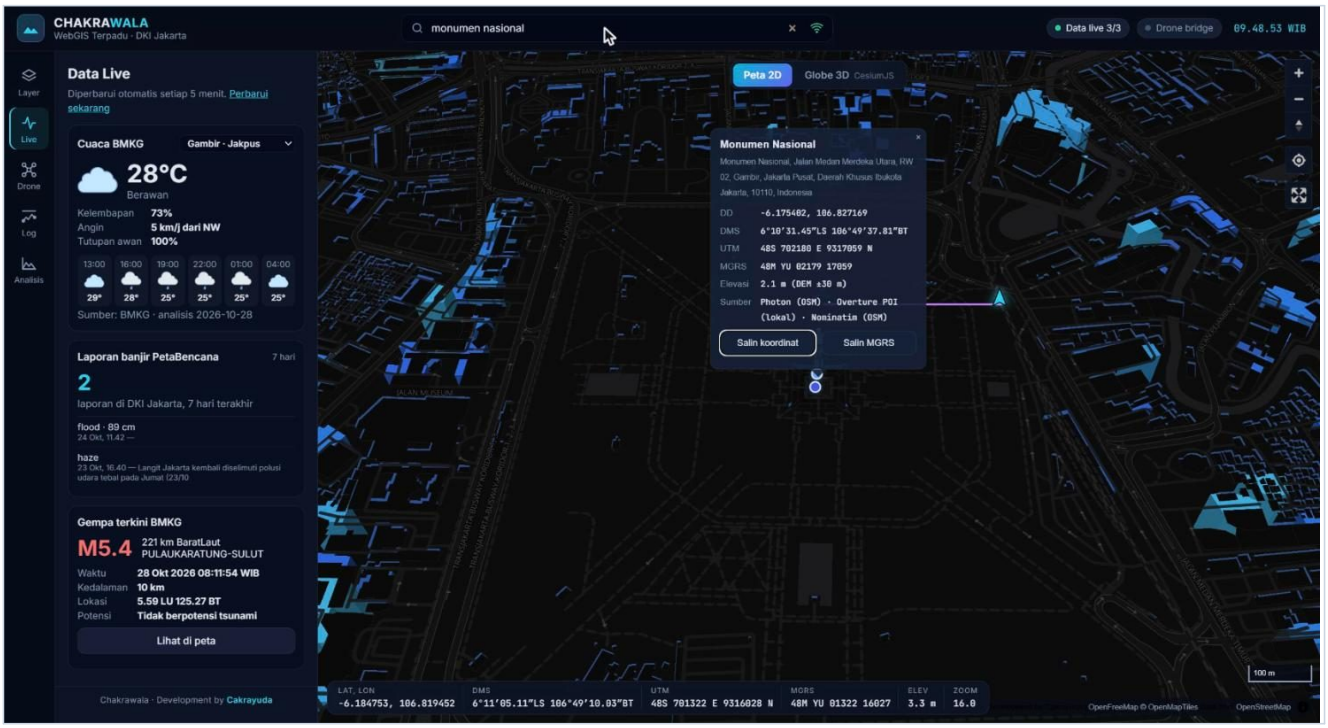


Data live dan hasil pencarian multi-provider

Langkah 3 — Hasil pencarian & salin koordinat

Memilih hasil akan menerbangkan peta ke lokasi dan membuka popup berisi DD, DMS, UTM, MGRS, elevasi, dan sumber data. Koordinat yang diketik (DD/DMS/UTM/MGRS) diproses lokal tanpa internet.

1. Pilih hasil teratas.
2. Klik **Salin koordinat** atau **Salin MGRS**.

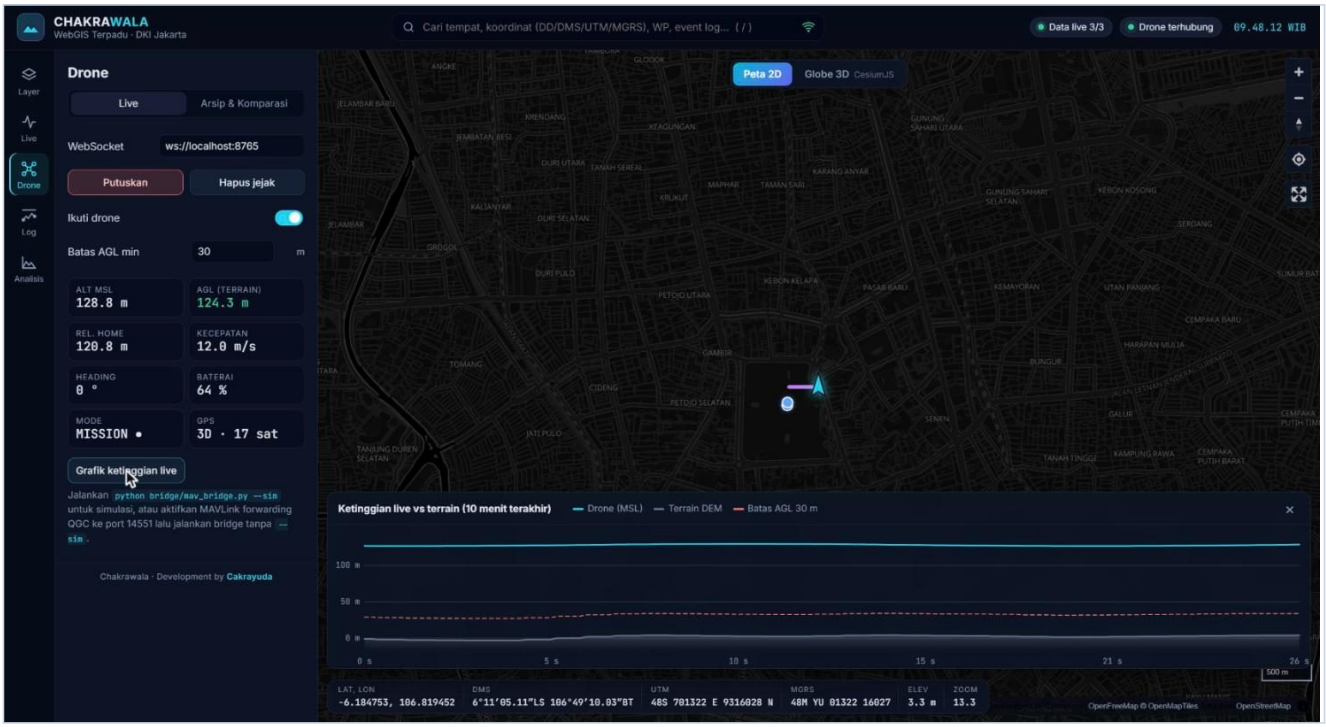


Popup hasil pencarian

Langkah 4 — Drone live (simulasi)

Bridge Python meneruskan telemetri MAVLink ke browser lewat WebSocket. Mode simulasi menerbangkan survei di atas Monas tanpa perangkat keras.

1. Jalankan `npm run bridge:install` lalu `npm run bridge:sim` di terminal terpisah.
2. Klik ikon **Drone** → **Live** → **Hubungkan**.
3. Klik **Grafik ketinggian live**; atur **Batas AGL min** sesuai kebutuhan.

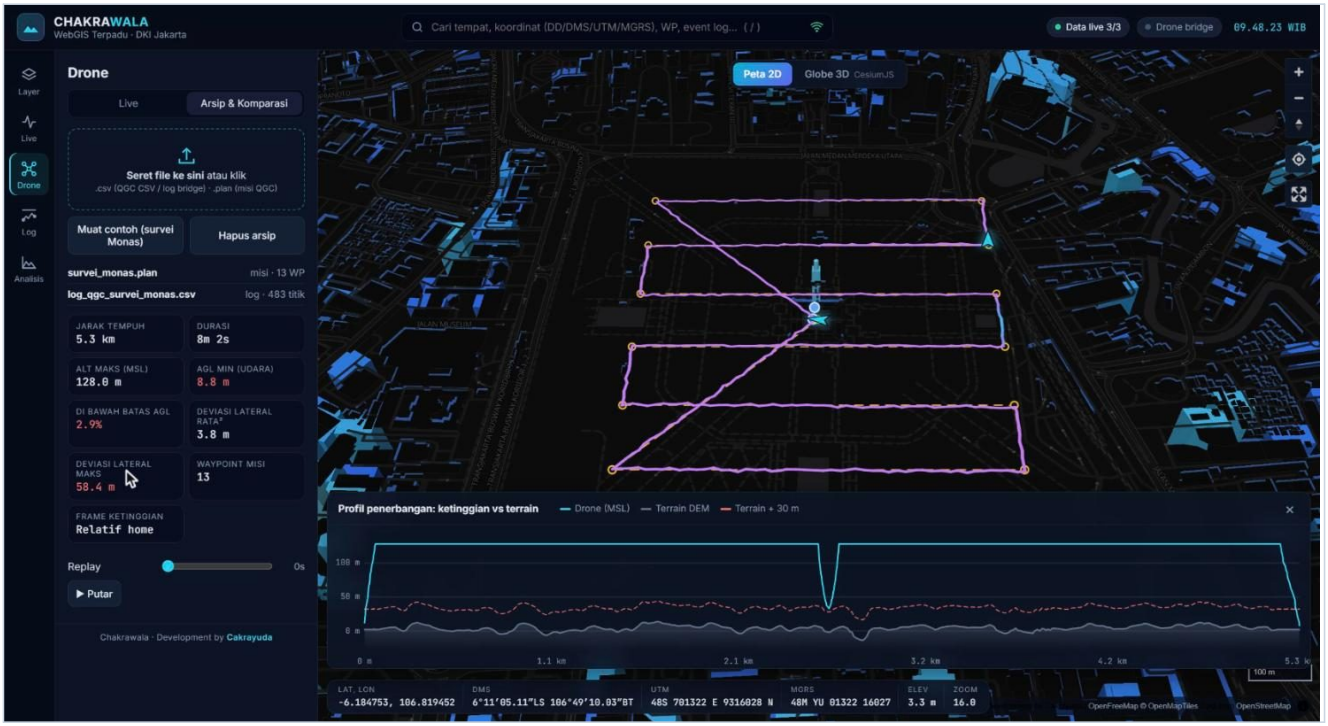


Drone live dengan HUD dan grafik ketinggian

Langkah 5 — Arsip CSV & rencana misi

Tab **Arsip & Komparasi** membandingkan log CSV QGroundControl dengan rencana misi: profil ketinggian, persentase di bawah batas AGL, dan deviasi lateral.

1. Klik **Drone** → **Arsip & Komparasi**.
2. Klik **Muat contoh (survei Monas)**.
3. Gunakan slider **Replay** atau tombol **Putar**.

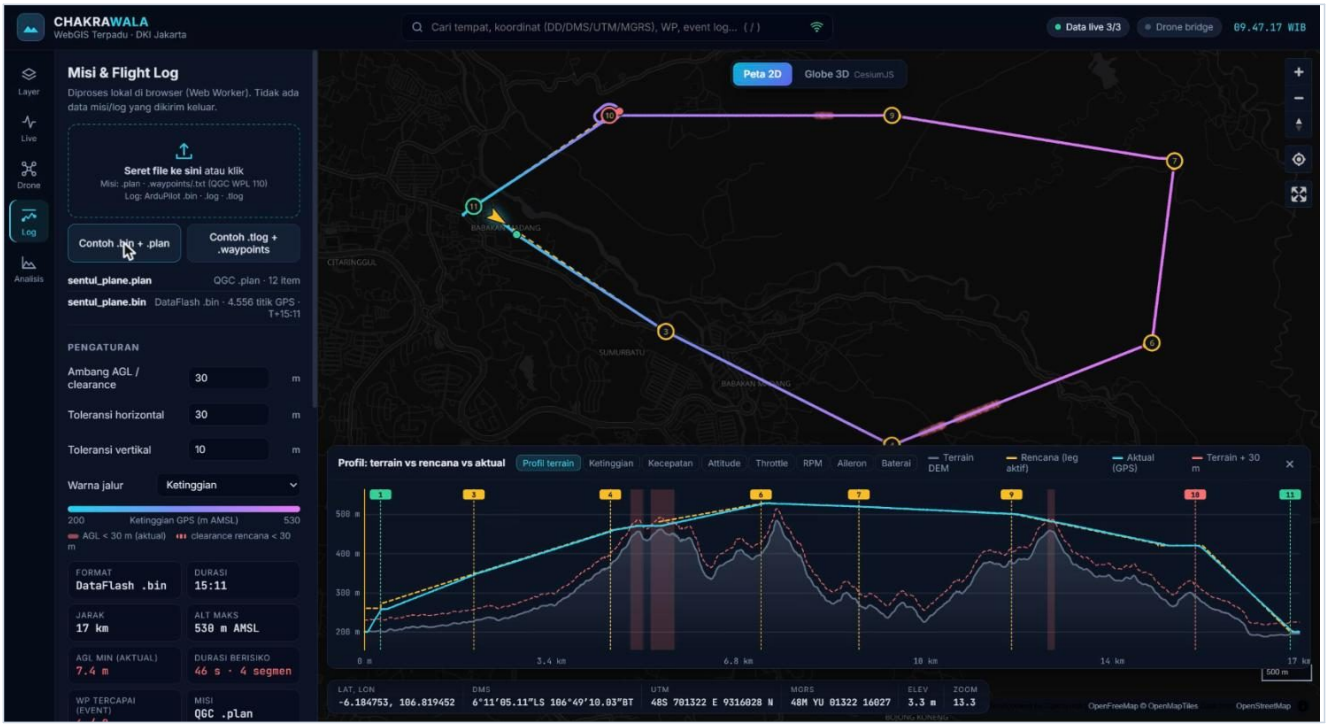


Arsip survei Monas: jalur, rencana, dan profil

Langkah 6 — Analisis misi & flight log

Menu **Log** menerima misi `.plan` / `.waypoints` / `.txt` dan log ArduPilot `.bin` / `.log` / `.tlog`. Semua file diproses di browser (Web Worker).

1. Klik ikon **Log**.
2. Klik **Contoh .bin + .plan** (atau seret file sendiri ke kotak unggah).
3. Baca kartu statistik: AGL minimum, durasi berisiko, WP tercapai, clearance minimum rencana.
4. Drawer **Profil terrain** menampilkan terrain DEM, ketinggian rencana, ketinggian aktual, dan pita segmen berisiko.

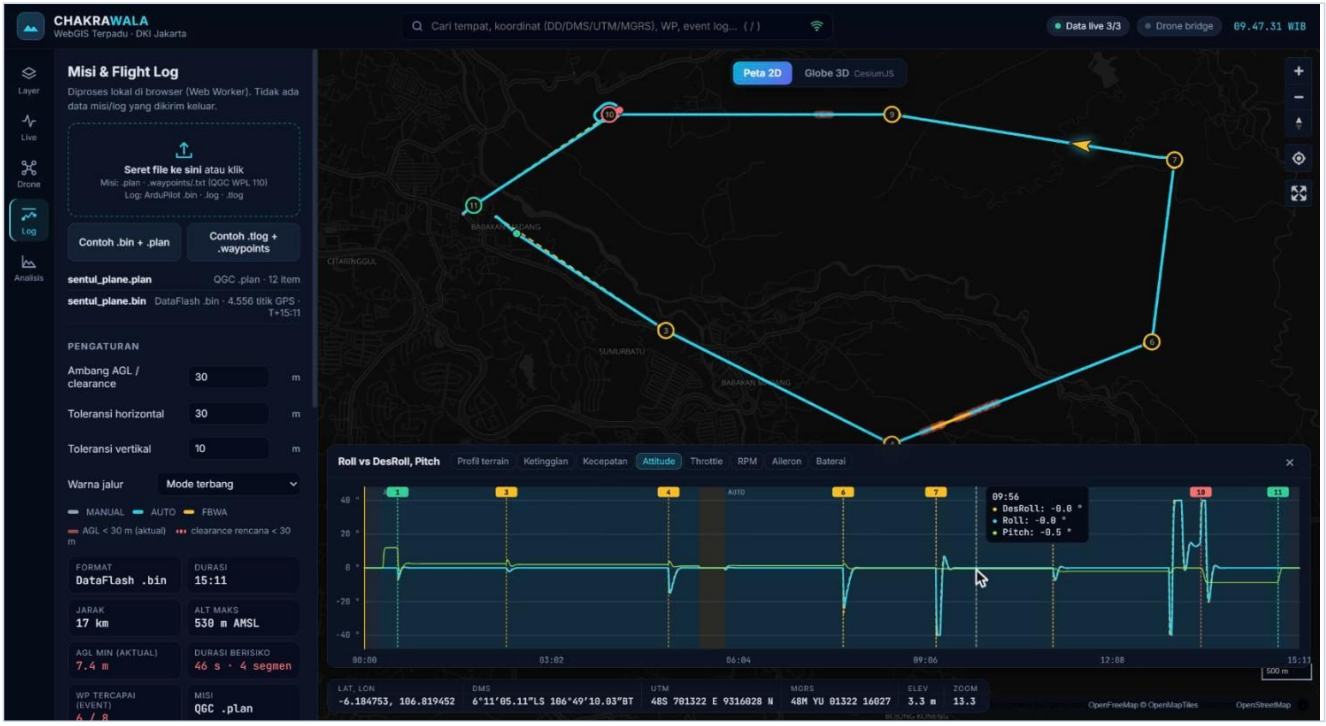


Analisis flight log sintetis

Langkah 7 — Grafik tersinkron & warna per mode

Semua grafik memakai satu cursor waktu yang tersinkron dengan ikon pesawat di peta dan readout di panel.

1. Ubah **Warna jalur** menjadi **Mode terbang**.
2. Pilih tab **Attitude** (tersedia juga Ketinggian, Kecepatan, Throttle, RPM, Aileron, Baterai).
3. Arahkan kursor di grafik; pita latar menunjukkan mode terbang.

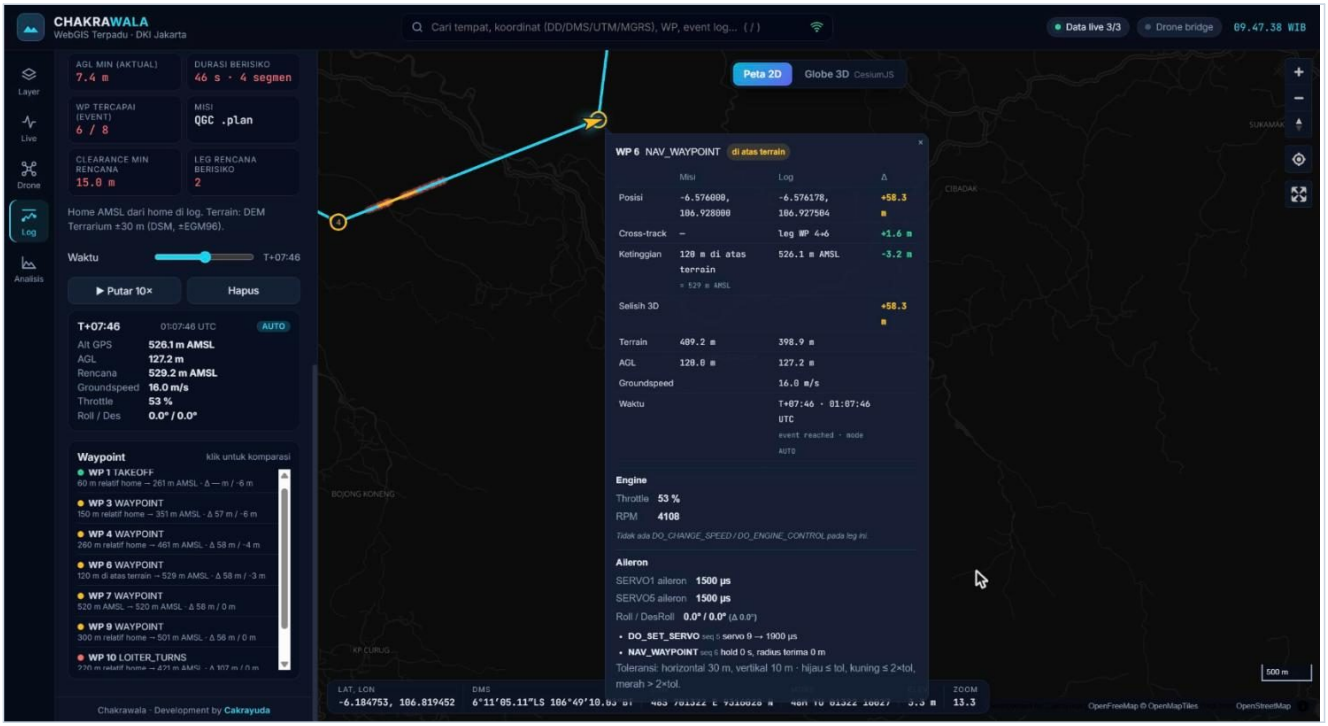


Attitude dengan pita mode terbang

Langkah 8 — Komparasi waypoint

Popup komparasi membandingkan rencana dengan kondisi aktual saat waypoint dicapai, termasuk engine dan aileron/elevon. Delta berwarna hijau bila dalam toleransi, kuning bila ≤ 2× toleransi, merah bila lebih.

1. Klik marker WP di peta, marker di grafik, atau baris di daftar **Waypoint**.
2. Atur **Toleransi horizontal** dan **Toleransi vertikal** di panel.

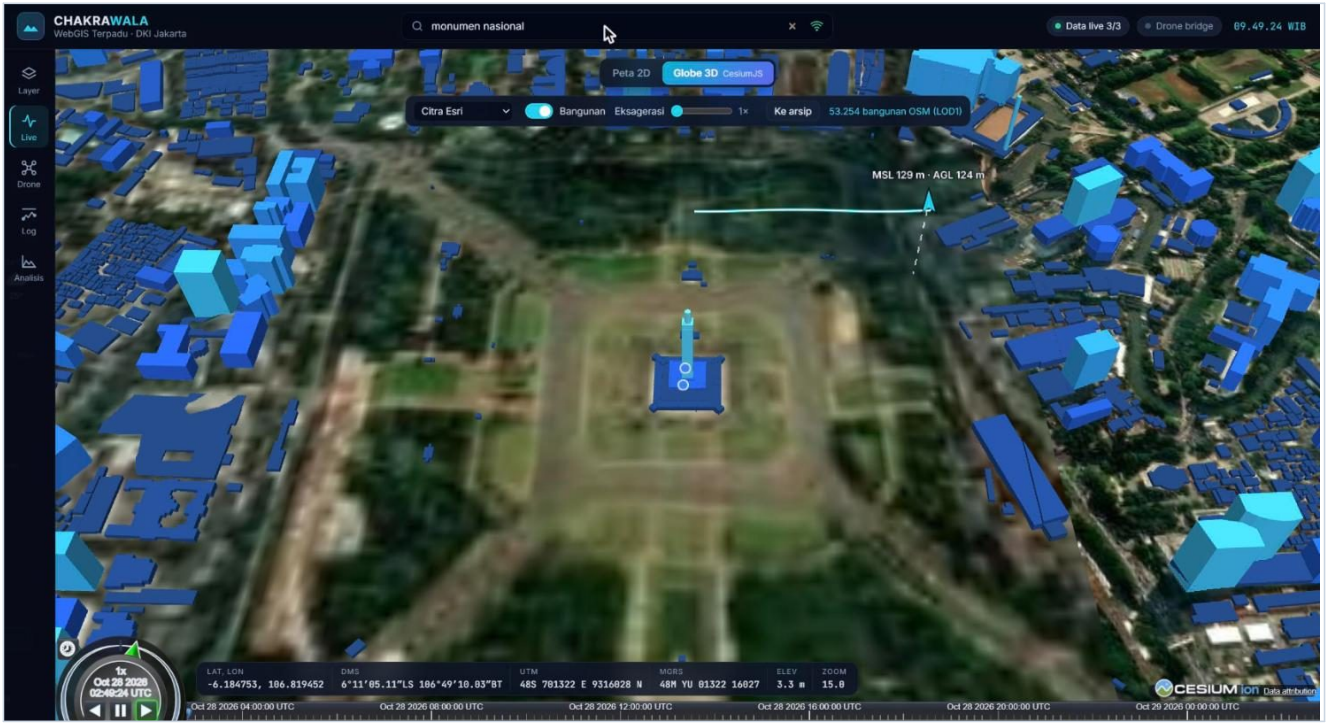


Popup komparasi waypoint

Langkah 9 — Globe 3D

Globe CesiumJS memakai terrain DEM yang sama dengan peta 2D dan mengekstrusi bangunan OSM (LOD1) tanpa token Cesium ion.

1. Klik **Globe 3D** di atas peta.
2. Atur basemap, sakelar **Bangunan**, dan **Eksagerasi**.
3. Klik **Peta 2D** untuk kembali — kamera tetap di lokasi yang sama.

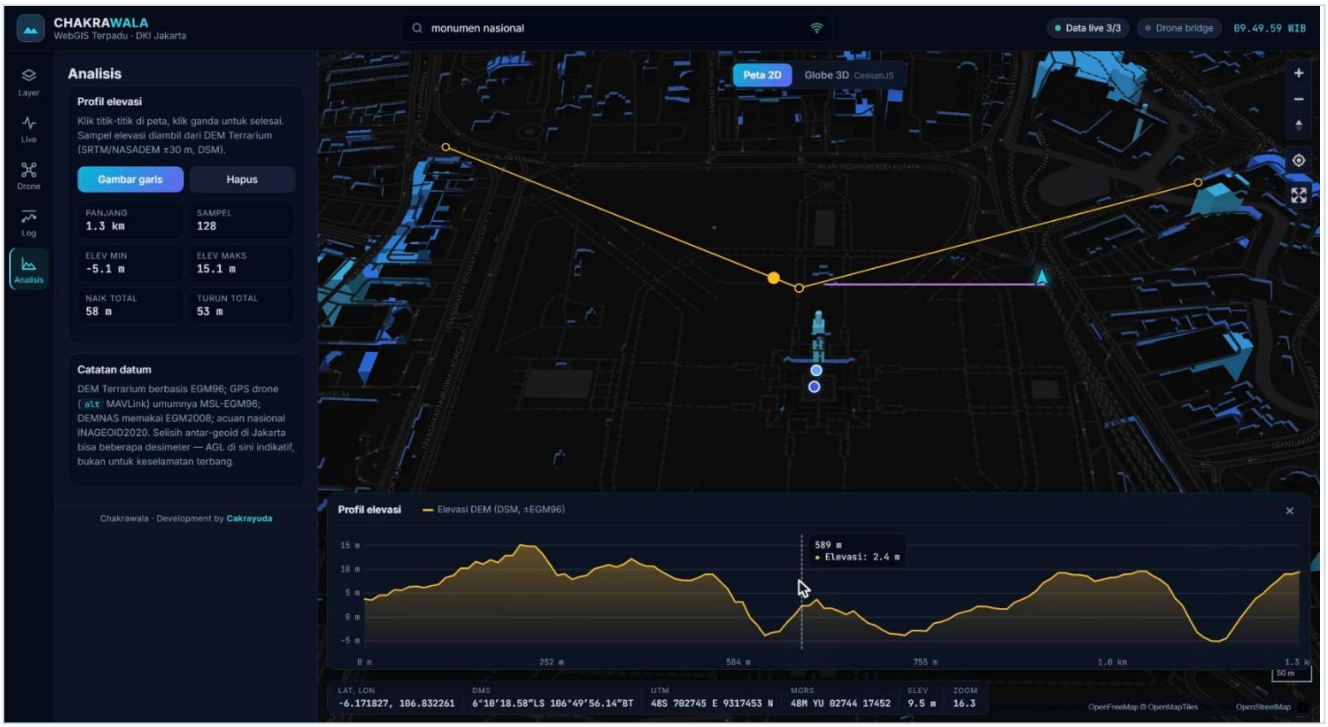


Globe 3D dengan jejak drone simulasi

Langkah 10 — Profil elevasi

Menu **Analisis** mengukur jarak dan profil elevasi DEM sepanjang garis bebas.

1. Klik ikon **Analisis** → **Gambar garis**.
2. Klik titik-titik di peta, klik ganda untuk selesai.
3. Arahkan kursor di grafik untuk melihat titik terkait di peta.



Profil elevasi

Catatan akurasi

- Elevasi berasal dari DEM Terrarium (SRTM/NASADEM ±30 m, DSM, ±EGM96). AGL bersifat indikatif dan **bukan** untuk keselamatan terbang.
- Ketinggian GPS (AMSL) dan DEM dianggap sama-sama EGM96; acuan nasional INAGEOID2020 dapat berbeda beberapa desimeter.
- Bila tipe wahana tidak terdeteksi dari log, aplikasi mengasumsikan ArduPlane dan menampilkannya di panel.

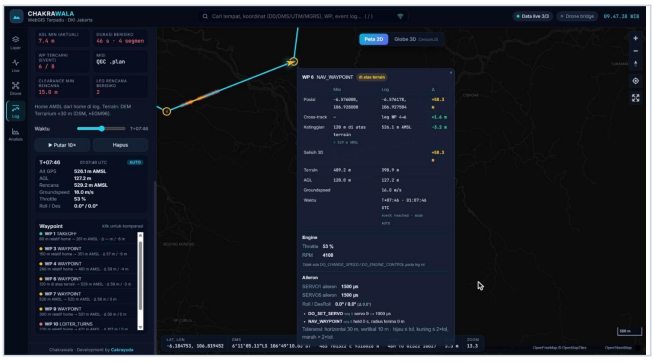
Referensi Teknis — Chakrawala

WebGIS terpadu untuk Jakarta — peta 2D/3D, data live, telemetri drone, serta analisis misi & flight log terhadap terrain.

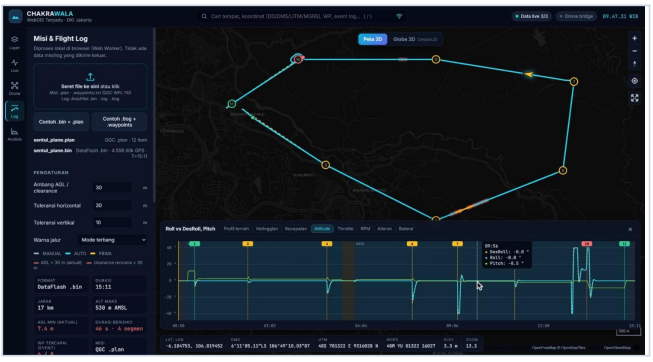
> Semua screenshot di dokumen ini dibuat dari **data demo sintetis** (`web/public/demo/`), bukan data penerbangan nyata.

Fitur

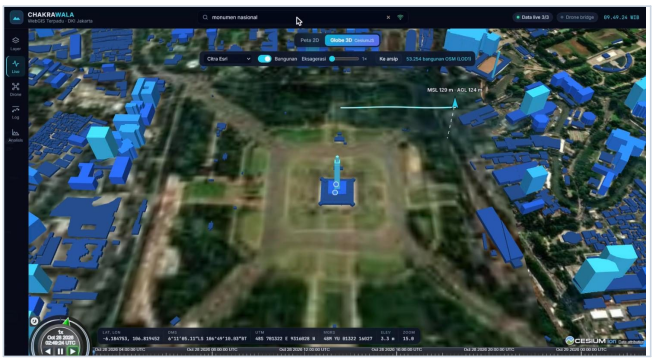
Area	Kemampuan
Peta 2D	MapLibre GL, basemap vektor gelap (OpenFreeMap/OSM), bangunan 3D, citra Esri & Sentinel-2, hillshade, terrain 3D, swipe komparasi citra. Setiap layer menampilkan sumber, resolusi, lisensi, dan **umur data**.
Koordinat	Kursor dalam DD, DMS, UTM 48S, MGRS, plus elevasi DEM.
Data live	Cuaca BMKG per kelurahan, laporan banjir PetaBencana, gempa terkini BMKG (refresh 5 menit).
Drone live	Jembatan MAVLink → WebSocket (Python). HUD MSL/AGL/kecepatan/baterai/mode/GPS, jejak berwarna, peringatan AGL, grafik live, mode simulasi.
Misi	Import QGC `.plan` dan Mission Planner `.waypoints`/`.txt` (QGC WPL 110) ke satu model ternormalisasi; frame `GLOBAL`, `RELATIVE_ALT`, `TERRAIN_ALT` diselesaikan ke AMSL.
Flight log	ArduPilot `.bin`, `.log` (teks), `.tlog` (MAVLink 1/2, CRC diverifikasi) di **Web Worker** dengan downsampling min-max.
Analisis	Profil terrain vs rencana vs aktual, AGL, clearance minimum, segmen berisiko (ambang bisa diatur), grafik time-series tersinkron dengan peta lewat cursor waktu.
Komparasi waypoint	Popup misi vs log: posisi, cross-track, selisih 3D/ketinggian, groundspeed, waktu; engine (throttle, RPM, `DO_CHANGE_SPEED`/`DO_ENGINE_CONTROL`); aileron/elevon (`SERVOn_FUNCTION`, Roll vs DesRoll, `DO_SET_SERVO`). Data yang tidak ada ditulis "tidak tersedia".
Pencarian	Antarmuka `GeocodingProvider`: Photon & Nominatim (default/fallback), POI Overture lokal, opsional Mapbox/MapTiler/HERE/Geoapify/Google Places. Query paralel, dedupe, ranking, sumber ditampilkan, mode offline. Input DD/DMS/UTM/MGRS diproses lokal.
Globe 3D	CesiumJS tanpa token ion: terrain DEM yang sama dengan 2D, bangunan OSM LOD1, drone live & replay arsip 3D.



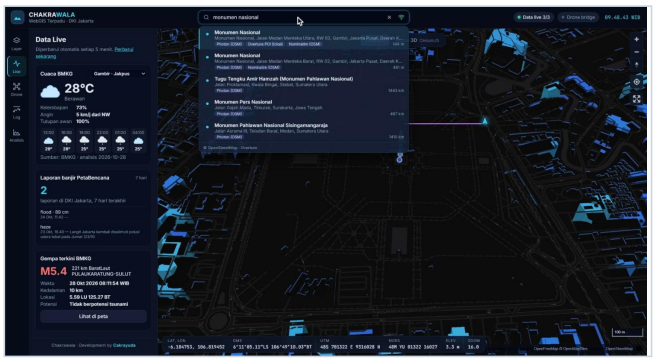
Popup komparasi waypoint (misi vs log)



Attitude & pita mode terbang, tersinkron ke peta



Globe 3D CesiumJS dengan jejak drone simulasi



Pencarian multi-provider dengan sumber

Arsitektur

Diagram arsitektur (versi interaktif Mermaid ada di README di GitHub). Komponen:

- Browser (Vite + vanilla JS)
- UI: panel, top bar, drawer grafik
- MapLibre GL peta 2D
- CesiumJS globe 3D (lazy)
- Web Worker parser .bin/.log/.tlog
- analysis.js AGL, clearance, komparasi WP
- geocode.js provider + dedupe + ranking
- Mesin lokal
- bridge/mav_bridge.py MAVLink → WebSocket
- QGroundControl / autopilot
- Layanan publik
- Tile peta, citra, DEM Terrarium
- BMKG · PetaBencana (via proxy Vite)
- Photon · Nominatim · provider berkunci (opsional)

Data misi dan log **tidak pernah** dikirim ke luar browser; yang dikirim ke provider geocoding hanya teks pencarian dan viewport.

Prasyarat

Komponen	Versi	Keterangan
Node.js	**22 LTS** atau lebih baru (lihat `.nvmrc`)	npm ikut terpasang
Python	3.10+	Hanya untuk bridge drone & alat bantu (opsional)
Browser	Chrome / Edge / Firefox terbaru dengan WebGL 2	Peta 2D & globe 3D
Git	apa saja	untuk clone

Berjalan di **Windows, macOS, dan Linux**; semua skrip npm memakai Node sehingga tidak bergantung pada shell tertentu.

Instalasi

```
git clone https://github.com/songohaji99/gcs-terrain-log-analyzer.git
```

```
cd gcs-terrain-log-analyzer
```

```
npm run setup
```

`npm run setup` menjalankan `npm ci` di folder `web/` (sesuai `package-lock.json`) dan menyalin aset CesiumJS ke `web/public/cesium/` secara otomatis.

Jalankan aplikasi:

```
npm run dev
```

Buka **http://localhost:5173**. Untuk build produksi:

```
npm run build
```

```
npm run preview
```

Bridge drone (opsional, Python)

Disarankan memakai virtual environment di folder proyek (`.venv`). Skrip npm otomatis memakai `.venv` bila ada, jadi **tidak perlu diaktifkan**.

macOS / Linux:

```
python3 -m venv .venv
```

Windows:

```
python -m venv .venv
```

Lalu, dari folder proyek:

```
npm run bridge:install
```

```
npm run bridge:sim
```

Mode live: di QGroundControl buka **Application Settings** → **MAVLink** → **MAVLink forwarding**, aktifkan dan arahkan ke `IP_KOMPUTER:14551`, lalu jalankan `npm run bridge:live`. Setiap sesi terekam ke `bridge/logs/flight_*.csv` (tidak ikut ke git).

Setup & konfigurasi API key

Semua key **opsional**. Tanpa key, pencarian memakai Photon + Nominatim (+ POI Overture bila ekstraknya dibuat).

1. Salin contoh konfigurasi:

```
node -e "require('fs').copyFileSync('web/.env.example','web/.env')"
```

2. Isi key yang dimiliki di `web/.env`:

Variabel	Provider	Catatan
`VITE_GEOCODERS`	—	Urutan/prioritas provider
`VITE_MAPBOX_TOKEN`	Mapbox Search	autocomplete
`VITE_MAPTILER_KEY`	MapTiler Geocoding	autocomplete

Variabel	Provider	Catatan
`VITE_HERE_API_KEY`	HERE Autosuggest	autocomplete
`VITE_GEOAPIFY_KEY`	Geoapify	autocomplete
`VITE_GOOGLE_PLACES_KEY`	Google Places (New)	hanya pencarian penuh
`VITE_PHOTON_URL`, `VITE_NOMINATIM_URL`	self-host	disarankan untuk produksi/offline
`VITE_NOMINATIM_EMAIL`	Nominatim publik	identitas kontak sesuai kebijakan OSMF
`VITE_OVERTURE_URL`	POI lokal	default `/data/overture_places.json`

3. Restart `npm run dev`.

> Variabel `VITE_*` ikut ter-bundle ke browser. Gunakan key yang **dibatasi per domain/referrer** di konsol provider. `web/.env` sudah di-ignore git — jangan commit key.

POI Overture untuk pencarian offline (opsional)

```
npm run tools:install
```

```
node scripts/py.mjs tools/overture_download.py --bbox 106.76,-6.26,106.90,-6.12
```

Hasilnya `web/public/data/overture_places.json` (di-ignore git, dibangun ulang di setiap mesin).

Penggunaan

Langkah	Cara
Coba tanpa data sendiri	Menu **Log** → **Contoh .bin + .plan** atau **Contoh .tlog + .waypoints**; menu **Drone** → **Arsip** → **Muat contoh**
Analisis misi	Seret `.plan` / `.waypoints` / `.txt` ke menu **Log**
Analisis log	Seret `.bin` / `.log` / `.tlog`; bila tanpa misi, misi diambil dari log
Grafik	Tab di drawer: Profil terrain, Ketinggian, Kecepatan, Attitude, Throttle, RPM, Aileron, Baterai
Komparasi waypoint	Klik marker WP di peta/grafik atau baris di daftar Waypoint
Pencarian	`/` untuk fokus, `↑/↓` pilih, `Enter` buka, `Shift+Enter` pencarian penuh, `Esc` tutup
Globe 3D	Tombol **Globe 3D** di atas peta

Data demo dibuat ulang dengan `npm run demo:generate` (butuh `npm run tools:install`).

Pengujian

```
npm test
```

```
npm run lint
```

```
npm run check
```

28 unit test (`node:test`) mencakup parser misi (`.plan` ≡ `.waypoints`), parser log (`.bin`/`.log`/`.tlog` dan file rusak), analisis (target leg, AGL, segmen berisiko, komparasi waypoint), parser koordinat, serta gabung/dedupe/ranking geocoding dengan provider di-mock. `npm run check` = lint + test + build.

Struktur proyek

```
.
├── web/                      # Aplikasi (Vite)
│   ├── index.html
│   ├── src/
│   │   ├── main.js           # peta, layer, koordinat, tab
│   │   ├── flightlog.js      # menu Log (misi + flight log)
│   │   ├── analysis.js       # AGL, clearance, komparasi waypoint
│   │   ├── mission.js        # parser .plan / WPL 110
│   │   ├── logparse.js       # parser .bin / .log / .tlog
│   │   ├── logworker.js      # Web Worker parser
│   │   ├── mavlink-defs.js   # di-generate dari pymavlink
│   │   ├── geocode.js        # GeocodingProvider + dedupe/ranking
│   │   ├── search.js         # UI pencarian
│   │   ├── cesium3d.js       # globe 3D
│   │   ├── drone.js, live.js, profile.js, chart.js, geo.js, elevation.js, parsers.js
│   │   └── *.test.js         # unit test
│   ├── public/demo/          # data demo SINTETIS (aman dipublikasikan)
│   ├── scripts/copy-cesium.mjs
│   └── .env.example
├── bridge/mav_bridge.py      # MAVLink → WebSocket (sim & live)
├── tools/                    # generator demo, defs MAVLink, Overture, PDF panduan
├── scripts/py.mjs            # runner Python lintas OS
├── docs/                     # screenshot, panduan PDF, riset
└── package.json              # skrip root (setup, dev, test, build, ...)
```

Privasi & data

- File misi/log diproses **di browser** (Web Worker); tidak ada unggahan ke server.
- `.gitignore` menolak semua `*.bin`, `*.log`, `*.tlog`, `*.ulg`, `*.plan`, `*.waypoints`, folder `samples/` dan `data/`, `.env`, cache DEM/tile, `node_modules`, `venv`, hasil build, serta file OS/IDE. Satu-satunya pengecualian adalah `web/public/demo/` yang berisi data **sintetis** hasil `tools/gen_samples.py`.
- Atribusi data pihak ketiga: © OpenStreetMap contributors (ODbL), OpenFreeMap/OpenMapTiles, Esri World Imagery (tampilan saja), EOX Sentinel-2 cloudless (CC BY-NC-SA 4.0), Mapzen Terrarium, BMKG, PetaBencana.id, Overture Maps.

Troubleshooting

Gejala	Solusi
`npm run setup` gagal karena versi Node	Pasang Node 22 LTS (`nvm use` membaca `.nvmrc`). Cek dengan `node -v`.
`EBADENGINE` / test tidak menemukan file	Node < 22 tidak mendukung glob pada `node --test`; upgrade Node.
Globe 3D gagal dimuat	Pastikan `web/public/cesium/` ada — jalankan ulang `npm run setup`.
Port 5173 sudah dipakai	Hentikan proses lain atau jalankan `npm --prefix web run dev -- --port 5174`.
"Data live 0/3"	Koneksi internet/proxy diblokir. Di server produksi, sediakan rute `/api/pb`, `/api/bmkg`, `/api/gempa` (lihat `web/vite.config.js`).
"Drone bridge" merah	Bridge belum berjalan: `npm run bridge:sim`. Aplikasi menyambung ulang tiap 3 detik.
Windows: `pip` gagal dengan `OSError ... Long Path` saat `bridge:install`	Clone ke folder yang lebih pendek (mis. `C:\src\`) atau aktifkan *Windows Long Path support* (lihat pesan pip).
`Python 3.10+ tidak ditemukan`	Pasang Python dan pastikan `python3` (macOS/Linux) atau `python`/`py` (Windows) ada di PATH.
Log ditolak: "tidak ada data GPS dengan fix 3D"	Log direkam tanpa fix GPS. Gunakan log dengan penerbangan bergerak.
`.ulg` ditolak	PX4 ULog belum didukung; gunakan log ArduPilot atau `.tlog`.

Gejala	Solusi
Pencarian kosong	Provider online gagal/rate limit — coba `Shift+Enter`, periksa ikon offline (kuning = offline).

Panduan pengguna (PDF)

Panduan langkah demi langkah tersedia di `docs/Chakrawala-Panduan-Pengguna.pdf`, dibangun dari dokumentasi ini dan screenshot sintetis:

```
npm run docs:pdf
```

Lisensi

Dirilis di bawah lisensi MIT. Hak cipta © 2026 Cakrayuda.